1.

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

df = pd.read_csv("data.csv")

df['Date'] = pd.to_datetime(df['Date'])

plt.figure(figsize=(8,4))
plt.plot(df['Date'], df['Sales'])
plt.title("Time Series Plot")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.show()

plt.figure(figsize=(8,4))
plt.plot(df['Date'], df['Sales'], marker='o')
plt.title("Seasonal Plot")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.show()

result = seasonal_decompose(
    df['Sales'],
    model='additive',
    period=12
)

result.plot()
plt.show()
```

2.

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

df = pd.read_csv("data.csv")

ts = df.iloc[:, 1]

plt.figure(figsize=(8,4))
plt.plot(ts)
plt.title("Time Series Plot")
plt.xlabel("Time")
plt.ylabel("Value")
plt.grid(True)
plt.show()

plot_acf(ts, lags=20)
plt.title("Autocorrelation Function (ACF)")
plt.show()

plot_pacf(ts, lags=20, method='ywm')
plt.title("Partial Autocorrelation Function (PACF)")
plt.show()
```

3.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import mean_absolute_error, mean_squared_error

df = pd.read_csv("data.csv")
y = df.iloc[:,1]

alpha = 0.3
s = [y[0]]

for i in range(1, len(y)):
    s.append(alpha*y[i] + (1-alpha)*s[i-1])

print("Smoothing parameter (Alpha) =", alpha)
print("MAE =", round(mean_absolute_error(y, s), 2))
print("RMSE =", round(np.sqrt(mean_squared_error(y, s)), 2))

plt.plot(y)
plt.title("Original Time Series")
plt.show()

plt.plot(y, label="Original")
plt.plot(s, label="Smoothed")
plt.title("First Order Exponential Smoothing")
plt.legend()
plt.show()
```

4.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error

df = pd.read_csv("data.csv")

ts = df.iloc[:, 1]

X = np.arange(len(ts)).reshape(-1, 1)
y = ts.values

model = LinearRegression()
model.fit(X, y)
pred = model.predict(X)

mae = mean_absolute_error(y, pred)
rmse = np.sqrt(mean_squared_error(y, pred))

diff_series = ts.diff().dropna()

log_series = np.log(ts)

print("Actual vs Predicted Values")
for i in range(5):
    print(f"Actual = {y[i]:.2f}   Predicted = {pred[i]:.2f}")

print("\nMAE =", round(mae, 2))
print("RMSE =", round(rmse, 2))

plt.figure(figsize=(8,4))
plt.plot(ts, marker='o')
plt.title("Original Time Series")
plt.xlabel("Time")
plt.ylabel("Value")
plt.show()

plt.figure(figsize=(8,4))
plt.plot(y, label="Actual")
plt.plot(pred, label="Predicted")
plt.title("Actual vs Predicted")
plt.xlabel("Time")
plt.ylabel("Value")
plt.legend()
plt.show()
```

```
plt.figure(figsize=(8,4))
plt.plot(diff_series, marker='o')
plt.title("Differenced Series")
plt.xlabel("Time")
plt.ylabel("Differenced Value")
plt.show()

plt.figure(figsize=(8,4))
plt.plot(log_series, marker='o')
plt.title("Log Transformed Series")
plt.xlabel("Time")
plt.ylabel("Log(Value)")
plt.show()
```

5.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error, mean_absolute_error

df = pd.read_csv("data.csv")

ts = df.iloc[:, 1]

plt.figure(figsize=(8,4))
plt.plot(ts)
plt.title("Original Time Series")
plt.show()

result = adfuller(ts)
print("ADF Statistic:", result[0])
print("P-Value:", result[1])

plot_acf(ts, lags=20)
plt.show()

plot_pacf(ts, lags=20)
plt.show()

n = int(len(ts) * 0.8)
train = ts[:n]
test = ts[n:]

model = ARIMA(train, order=(1,1,1))
model_fit = model.fit()

pred = model_fit.forecast(steps=len(test))

rmse = np.sqrt(mean_squared_error(test, pred))
mae = mean_absolute_error(test, pred)

print("RMSE:", rmse)
print("MAE:", mae)

plt.figure(figsize=(8,4))
plt.plot(test.values, label="Actual")
plt.plot(pred.values, label="Predicted")
plt.title("ARIMA Predicted vs Actual")
```

```
plt.legend()
plt.show()
```

6.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.statespace.sarimax import SARIMAX
from sklearn.metrics import mean_absolute_error, mean_squared_error

df = pd.read_csv("data.csv")

ts = df.iloc[:, 1]

train_size = int(len(ts) * 0.8)
train = ts[:train_size]
test = ts[train_size:]

model = SARIMAX(train,
                order=(1,1,1),
                seasonal_order=(1,1,1,12))

result = model.fit()

forecast = result.forecast(steps=len(test))

mae = mean_absolute_error(test, forecast)
rmse = np.sqrt(mean_squared_error(test, forecast))
mape = np.mean(np.abs((test - forecast) / test)) * 100

print("MAE :", round(mae,2))
print("RMSE:", round(rmse,2))
print("MAPE:", round(mape,2), "%")

plt.figure(figsize=(8,4))
plt.plot(ts)
plt.title("Original Time Series")
plt.show()

plt.figure(figsize=(8,4))
plt.plot(test.index, test, label="Actual")
plt.plot(test.index, forecast, label="Forecast")
plt.title("SARIMA Actual vs Forecast")
plt.legend()
plt.show()

future_forecast = result.forecast(steps=12)

print("\nFuture 12-Month Forecast:")
print(future_forecast)
```

```
plt.figure(figsize=(8,4))
plt.plot(range(1,13), future_forecast, marker='o')
plt.title("Future 12-Month Forecast")
plt.xlabel("Month")
plt.ylabel("Forecast Value")
plt.show()
```